

Content and User Preference Based Music Recommendation System Using Knowledge Graphs and GNN

Group 7: Pedro Fanica (54346) Aarthi Perumpillichira (66404) Maja Skwarón (66083)

Abstract

We present a content- and preference-aware music recommendation system that couples a music knowledge graph (KG) with graph representation learning. We integrate the Lakh MIDI Dataset and the Million Song Dataset, extract 1,526 symbolic descriptors per track with jSymbolic, and construct an RDF/OWL knowledge graph that links users, tracks, artists, genres, instruments, keys, modes, tempo classes and release decades, with the upper ontology grounded in Wikidata. On top of this graph we train a stack of complementary models: a symbolic-feature autoencoder that compresses raw descriptors into dense track embeddings, two knowledge-graph embedding models (RotatE and ComplEx) that encode the relational structure, and a Heterogeneous Graph Transformer (HGT) trained with a popularity-debiased listwise objective that serves as our main recommender. We compare against three baselines—a popularity prior, user–user KNN collaborative filtering ($k=6$), and a two-stage gradient-boosted content/CF hybrid—on a shared stratified user-level split of 284,864 users and 6,054 tracks (1.94M training interactions). Results at $K=10$ show that KNN-CF achieves NDCG of 0.106 and 91.5% catalogue coverage with a composite Overall score of 0.420, tripling the popularity baseline (0.134) through diversity gains rather than raw accuracy improvements. The XGBoost hybrid reduces popularity bias to 0.032 (lowest of any model) but at a large accuracy cost (NDCG 0.015), highlighting the limits of content-only candidate generation without collaborative signals. **[TODO: Summarise the HGT headline result once training is complete.]**

1. Introduction

This project investigates how a Graph Neural Network (GNN)-based music knowledge graph, combining musical features (e.g., pitch statistics and tempo) as node embeddings, song metadata (such as artists and genres) as graph structure, and user interaction history as supervision, can be used to predict user-specific music prefer-

ences.

Although many music recommendation algorithms already exist and are widely used, most of them rely primarily on user interaction data and similarity measures between users. While suggestions based on mutual listening history overlap can be quite effective, this approach remains inherently limited and strongly favors music and artists that are already popular. Musical preference is highly subjective, and relying solely on behavioral patterns may overlook important aspects of the music itself.

In reality, musical pieces are a rich source of information, differing from one another across multiple dimensions. Each piece can be characterized by a wide range of features, from more direct attributes such as the artist or genre to strictly musical properties derived through analysis, such as pitch statistics, tempo, or chord progressions. Incorporating these musical features into the recommendation process may provide a richer and more structured representation of songs, enabling models to capture deeper relationships between musical content and user preferences.

Because music is both an art form and a structured domain, it is particularly well-suited for representation as a knowledge graph. By leveraging a knowledge graph in combination with Graph Neural Networks (GNNs), this project aims to model complex, multi-relational dependencies and explore their potential for innovative, more accurate and context-aware music preference prediction.

2. Data

2.1. Data Sources

This project integrates data from the two main sources:

- **Lakh MIDI Dataset (LMD)**[6] - A large-scale collection of MIDI files (machine-readable representation of musical structure) representing music in symbolic form (notes, timing, instruments).
- **Million Song Dataset (MSD)**[4][7] - A dataset developed by The Echo Nest, containing audio-derived features such as tempo, key, or loudness metadata for songs, as well as a user taste profile subset.

2.2. Data Processing

The first step of the data processing methodology involved the extraction and linking of the Lakh MIDI and Million Song Dataset (MSD). The alignment between MIDI files and audio tracks was based on precomputed similarity scores obtained using Dynamic Time Warping (DTW) by Raffel (2016). In this process, both MIDI and audio signals were first converted into chroma-based representations over time, and then compared to determine their similarity. The threshold for acceptable similarity score of 0.55 was set during the filtering step, in order to ensure reliable alignment. The remaining files and duplicates were omitted resulting in a dataset of 19,037 tracks, with each one described at this stage by 40 metadata fields which included track-level identifiers (e.g. track_id, song_id, or artist_id), descriptive fields (e.g. title, artist name, album, release year), tonal estimates (key, mode, and their confidences), rhythmic/audio scalars (time signature, loudness), genre/tag information (e.g. primary genre, top-3 genres), and MIDI-derived instrumentation (number and names of instruments). The exploratory overview of the MSD features is available in Figure 6 of the Appendix.

The next step concerned processing and subsequently linking user interaction data extracted from Echo Nest Taste Profile, represented in the form of (user_id, song_id, play_count) triplets. The raw dataset consisted of approximately 48 million interactions across over one million users and almost four hundred thousand songs. Interactions associated with known erroneous song ID mappings were removed using the official MSD mismatch list, which identified 18,913 incorrectly linked song IDs, of which 6,237 were present in the taste profile, accounting for 5.33% of all interactions. Subsequently, the user data was filtered to retain only the songs that also exist in the Lakh/MSD previously matched dataset, which constituted approximately 40.4% of all LMD songs. Additionally, instances with fewer than 5 remaining song interactions were removed to address the cold start problem, excluding users with insufficient listening history to train and evaluate recommendations for them. After the filtering step the dataset consisted of 299,156 unique users and 7,227 unique songs. Figure 1 illustrates the filtering funnel across three dimensions: total interactions, unique users, and unique songs, at each step of the filtering pipeline.

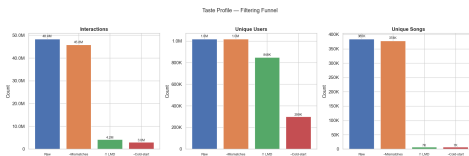


Figure 1. Taste Profile filtering funnel.

The detailed filtering statistics across all steps, together with filtered dataset diagnostics are summarized in Table 3 and 7 of the Appendix.

After the data integration and filtering process, the subsequent step included feature extraction and selection. Since raw MIDI files do not directly provide numerical descriptors suitable for machine learning, each track had to be converted into a fixed-length feature vector capturing its musical characteristics. For this purpose the jSymbolic feature extractor [5] was used. It works directly with symbolic representations of music and generates a wide range of high-level descriptors based on key concepts of music theory, making it particularly useful for extracting meaningful and interpretable features from MIDI data.

Because of the substantial data size, the extraction was implemented using batches of 100 songs at a time, which produced XML files containing the computed feature values together with a feature-definition schema. The resulting each XML output was subsequently converted into a tabular CSV representation, and each row was associated with its corresponding song_id by matching the MIDI file hash against the metadata table assembled in the previous step. Out of the 19,037 matched tracks, 7,050 were successfully processed constituting the basis for all downstream analysis and modelling.

For each track jSymbolic produced 1,525 numerical descriptors covering seven principal categories:

- **The pitch features** - describing the distribution and variety of notes used, including the basic pitch histogram, the pitch-class and folded-fifths histograms, the pitch range, and most-common-pitch statistics.
- **The melodic features** - characterizing tune movement and its direction through for example the melodic-interval histograms, or the direction and length of melodic arcs.
- **The vertical (harmonic) features** - representing pitch distances between notes played simultaneously.
- **The rhythmic features** - described by attributes such as beat histograms, tempo statistics or note density.
- **The dynamics features** - capturing the loudness range and the note-to-note dynamic level variation.
- **The textural features** - describing the polyphonic structure of the pieces, including the average number of independent voices, voice equality across several dimensions, and voices overlap.
- **The instrumentation features** - describing the statistics about instrument usage across each piece.

It is important to mention that the resulting high dimensionality of the feature space is largely due to the extracted histogram-based descriptors, which expand into one column per bin. The remaining features are scalars and there are around 150 of them for each track.

3. Methodology

3.1. Knowledge Graph Design and Construction

One of the main challenges and core components of this project was designing and populating the knowledge graph that would be used to train the recommendation algorithm. To support this objective, the knowledge graph must have been designed carefully to accommodate both diverse musical attributes and user behavior, as well as the relational structure necessary for effective representation learning.

The knowledge graph was built on top of an existing music ontology (ref), with classes and properties tailored to music recommendation system. Standard vocabularies expressed in RDF such as FOAF for agents, or event ontology for listening events were reused to ensure the graph is interoperable with other Semantic Web tools and its structure is transparent to readers familiar with these standards. A small number of project-specific relations were added under a dedicated `mrc:` namespace to cover concepts not present in the existing vocabularies, such as the link between a performance and its tempo category, or between a listening event and its play count.

3.1.1. Simple Knowledge Graph

The knowledge graph was built from accurately matched tracks described by both MSD metadata and successfully extracted `jSymbolic` features. Not all metadata fields available in the merged dataset were retained for the graph, but only those mapping onto a meaningful ontology concept. The features were carefully selected to ensure that every retained field could be expressed as either a node, an edge, or an interpretable property within the ontology (e.g. artist and track identifiers, title, album, release year, genre tags, or MIDI instrumentation). Derived audio descriptors such as danceability and energy, or popularity scores were excluded from the graph but remained available in the underlying tabular dataset for use in the subsequent modelling and evaluation stages, ensuring that the graph contains only information that contributes to the relational structure used by the recommender. Out of 40 columns produced by the MSD-Lakh integration only 22 were kept for the KG.

Similarly, out of the 1,525 `jSymbolic` features extracted per track, only 30, were initially selected as domain relevant, out of which 11, that cleanly mapped to ontology properties (e.g., `mrc:meanTempo`, `mrc:noteDensity`), were folded into the graph as datatype properties on the corresponding Track node. All selected features represented a single, well-defined musical attribute such as mean tempo, note density, amount of staccato, or average number of independent voices. The remaining 19 selected features, while musically interpretable, did not have a matching ontology predicate defined and were therefore retained only in the tabular dataset. The rest of the extracted `jSymbolic` features almost entirely consisted of multi-bin histograms such as

pitch, melodic interval, beat, rhythmic value, or instrument-prevalence histograms, whose individual bins did not correspond to musically meaningful concepts, therefore they were excluded. Such decision was made because including these features would clutter the graph without contributing much usable relational information. Again, the full feature vectors were saved and reserved as input to the autoencoder described in the following section, which subsequently compressed them into compact track-level embeddings used to initialize track nodes.

In the knowledge graph, each row of the merged dataset translates into a connected cluster of nodes representing the entities associated with that recording. Categorical and relational concepts such as genres, instruments, musical keys, modes, and tempo classes were encoded as named individual nodes reused across tracks, enabling the graph to capture shared structure between recordings. Continuous scalar measurements, unique to each track such as tempo, note density, or loudness were, on the other hand, encoded as datatype literals attached to the corresponding track or performance node.

In our knowledge graph, a Track node represents the song itself and carries basic descriptive attributes (title, song identifier, album, release year) as well as 11 `jSymbolic` datatype properties such as note density, chromatic motion, or chord duration. An Artist node represents the performer with name and MusicBrainz identifier. The two are linked through a Performance node, which acts as a bridge and stores the mean tempo value, following the design of the Music Ontology. Key (one of the twelve chromatic keys (e.g. C# or D)) and mode (one of the two modes (Major or Minor)) are attached directly to the Track node, each pointing to a controlled-vocabulary named individual, a fixed set of predefined, standardized values that were instantiated once and are shared across all tracks, rather than literal values. Each Performance node is additionally linked to one of nine TempoClass individuals derived from the mean tempo value (e.g. Andante or Allegro), and one or more Instrument nodes extracted directly from the MIDI files. Genre nodes are attached at the artist level rather than the track level, reflecting MSD's artist-level tagging. Additionally, each unique genre label was created once and reused across all artists associated with it, ensuring consistency, reducing redundancy, and enabling more effective querying and reasoning over musical properties.

This proposed design ensures that the graph captures both the relational structure between entities (artists, tracks, performances, genres, and instruments) and the musical properties of each track in a form that is directly exploitable by graph neural networks, where shared nodes such as genre or tempo class create meaningful connections between otherwise unrelated tracks and artists, enabling the downstream model to learn from structural proximity rather than feature

similarity alone.

The ontology was further enriched by aligning its domain classes to the Wikidata upper-concept hierarchy using OWL semantics. Specifically, `mrc:Genre` was declared as `owl:equivalentClass` to the Wikidata entity for music genre (`wd:Q188451`), `mo:Instrument` to musical instrument (`wd:Q34379`), and `mrc:Key` to key in music theory (`wd:Q534932`). These classes were additionally linked upward via `rdfs:subClassOf` to a shared parent `mrc:ElementOfMusic`, which was itself declared equivalent to the Wikidata concept of elements of music (`wd:Q11696608`), sitting under the broader musical concept node (`wd:Q115211517`). The two mode individuals (Major and Minor) were further grounded via `owl:sameAs` links to their respective Wikidata entities. Linking the ontology classes to Wikidata’s upper-concept hierarchy provided semantic grounding for the graph’s core musical concepts. Rather than treating genre, instrument, key, and tempo class as isolated, project-specific labels, the alignment via `owl:equivalentClass` and `rdfs:subClassOf` positions them within a universally recognized framework. Additionally, this design allows genre, key, tempo class, and instrument nodes to be shared across all tracks, acting as bridges in the graph. This shared node structure enables message passing to propagate musical context across otherwise disconnected parts of the graph, allowing the GNN model to learn from structural proximity between tracks and artists that share common musical properties, directly benefiting our model.

Because a year attribute is a poor feature on its own, being too sparse for any single value to carry useful signal, and carrying no structural notion of proximity in time, every observed year in our data was turned into its distinct decade of release (1960s through 2010s) and exposed as a first-class node. Decades were then linked into a sequence using `wdt:P155` (follows) and `wdt:P156` (followed by), and each local decade was further matched to its Wikidata decade entity and the parent century. This way hundreds of tracks started sharing the same nodes, and the year attribute was converted into something the graph and the embedding model can effectively exploit.

Lastly, the behavioral layer of the graph (users and listening events) from the Echo Nest Taste Profile dataset was added to the graph. Only rows whose `song_id` matched a track already present in the graph were kept, so that every listening link pointed to an existing node. Each unique user was represented as a single `mrc:Listener` node (a subclass of `foaf:Agent`), identified by their anonymised Echo Nest user ID stored as both a label and a `dc:identifier`. Each listening interaction, on the other hand, was encoded as a direct edge (`mrc:listenedTo`) from the user node to the track node, without recording how many times the song was played.

A summary of user-related graph components introduced

in the behavioral layer is available in Tables ?? in the Appendix.

3.1.2. Rich version of the Knowledge Graph

On the top of the presented knowledge graph, a second, richer version of it was additionally created that preserves extra details such as weights, counts, and confidence, preserving all the information from the source dataset. This version matches the Music Ontology’s intended modelling much more closely, and allows for more advanced SPARQL inspection and qualitative evaluation, using KG embedding methods that can handle heterogeneous edges, and for frequency-aware recommendation.

In this version Genre edges carry weights, with edges becoming small intermediate nodes (blank nodes) that carry the strength of the genre tag taken either from MSD’s published `artist_terms_weight` or, when missing, from a rank-based fallback. Key and mode are attached to the Performance instead of Tracks and carry confidences, creating a conceptually cleaner version, and allowing for evaluation of how certain the analysis is. Lastly, the play count is represented as a property of the event, allowing for the recommender to treat repeated listening as a stronger preference signal than a single play.

Because when using a blank node, some of the original relationships become a two-hop path (e.g. Artist → Assoc → Genre), one-layer GNN models can’t traverse it. For this reason only a simple version of the graph will be used as input to the HGT recommender. The richer version on the other hand, will be used to train the two knowledge-graph-embedding baselines, and additionally, will be treated as the canonical artifact for qualitative inspection.

The complete feature inventory of the enriched knowledge graph, as well as the full schema overview are available in section C of the appendix.

Figure 2 shows the prototype of the knowledge graph linking users, songs, and musical features, which forms the foundation for the subsequent recommendation model.

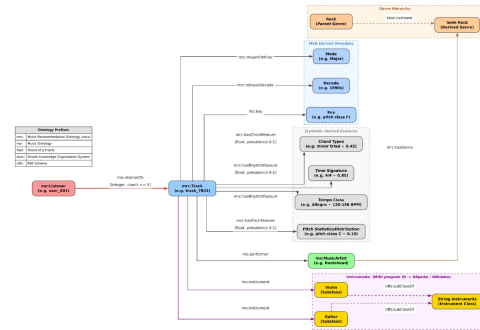


Figure 2. Graphical overview of the proposed methodology.

The graph will be populated by integrating the filtered

user interaction data with the Lakh/MSD dataset and the extracted musical features, using shared identifiers such as song ID and MIDI file mappings. This unified representation will enable the combination of relational, contextual, and musical information, forming the foundation for subsequent graph-based learning and recommendation models.

3.2. From Knowledge Graph to Learnable Tensors

Before any neural model can consume the graph, the RDF triples are converted into a typed tensor representation. A dedicated extraction routine streams the graph into (i) a consolidated triple table and an edge dictionary, and (ii) a heterogeneous PyTorch Geometric graph in which every ontology class becomes a distinct node type (user, track, artist, genre, instrument, key, mode, tempoClass, decade) and every predicate a typed relation. Track nodes are initialised with the content embeddings produced by the autoencoder described below; the remaining node types receive either knowledge-graph-embedding features or learnable lookup vectors. Because the listening layer of the simple graph encodes each interaction as a direct `user` $\rightarrow$ `track` edge, message passing can reach items in a single hop, which is the configuration used to train the main recommender. Crucially, the graph fed to the HGT is materialised from the *training* interactions only, so neither message passing nor the training objective can observe held-out edges. Validation and test scores are then computed against the held-out interactions of the *same* user-level stratified split that drives the baselines (Sec. 3.5), rather than an independent random edge split internal to the model. This makes the HGT directly and fairly comparable to the popularity, KNN-CF and hybrid baselines on identical folds.

3.3. Representation Learning and Recommendation Models

Our system combines three trained representation/recommendation components with three baselines. All models share the same preprocessed interaction data and the same evaluation split (Sec. 3.5).

Symbolic-feature autoencoder. The full 1,526-dimensional jSymbolic vector of each track is standardised and passed through a fully connected autoencoder with encoder $1526 \rightarrow 512 \rightarrow 256 \rightarrow 128$ and a symmetric decoder ($128 \rightarrow 256 \rightarrow 512 \rightarrow 1526$), each layer followed by ReLU activation and dropout 0.2. Training uses Adam (learning rate 10^{-3} , batch size 256) to minimise mean-squared reconstruction error, with up to 200 epochs and early stopping once the training loss fails to improve by more than 10^{-4} for 15 consecutive epochs. Because the task is pure transductive compression of a fixed catalogue, no held-out ranking split is required; the model converged

in 155 epochs (best MSE 0.616). The 128-dimensional bottleneck is exported as a dense *content* embedding for every track and is used both to initialise the `track` node features of the graph models and as the item representation of the hybrid baseline.

Knowledge-graph embeddings (structural/reasoning view). To encode the multi-relational structure of the enriched graph independently of any audio content, we train two knowledge-graph embedding (KGE) models with PyKEEN: RotatE [1] and ComplEx [2]. **[TODO: Replace these citation keys with the real RotatE/ComplEx references in main.bib.]** Triples are exported with the listening events removed from the training corpus (a *semantic* scope), so that held-out interactions cannot leak into the entity embeddings. Each entity is represented by a 128-dimensional complex vector, i.e. a 256-dimensional real vector $[\Re \parallel \Im]$ after flattening, and both models are trained for 200 epochs with Adam (learning rate 10^{-2} , batch size 1024) on the full triple set. Because the goal is to obtain embeddings for every entity in the graph rather than to predict held-out links, all triples are used for training. The entity embeddings provide a structure-only view of the catalogue that is used to seed node features and to support the qualitative latent-space analysis in Sec. 4.

Heterogeneous Graph Transformer (main model). Our principal recommender is a **Heterogeneous Graph Transformer (HGT)** [3] operating on the typed graph. **[TODO: Confirm/replace the HGT citation key.]** Each layer performs type-aware, attention-based message passing so that musical context propagates across the shared genre, key, tempo-class and decade nodes that bridge otherwise disconnected tracks. We use a hidden dimension of 128, 4 attention heads, 2 layers and dropout 0.1, and train for 100 epochs with Adam (learning rate 10^{-3} , weight decay 10^{-4}). Recommendations are scored as the inner product between the final user and track embeddings, with training-seen items masked. To counter the popularity bias that dominates interaction-only methods, the model is trained with a *popularity-debiased listwise* objective: a softmax cross-entropy over sampled items in which a log-popularity prior is subtracted from the logits (regularisation strength $\lambda = 0.2$, temperature 0.1). The checkpoint with the best validation Recall—measured on the shared stratified validation ground truth described above—is retained for testing.

Baselines. Three baselines contextualise the graph models. (i) **MostPopular** ranks unseen items by global play frequency. (ii) **KNN-CF** is a user–user collaborative filter computed on the ℓ_2 -normalised interaction matrix (cosine similarity); the neighbourhood size k is selected by a sweep

that maximises the composite validation score. **(iii) XGBoost two-stage hybrid** first retrieves the top-200 candidate tracks per user via the cosine product between a mean-pooled autoencoder *user profile* and every track’s 128-dimensional content embedding. These candidates are then re-ranked with a gradient-boosted learning-to-rank model (rank:ndcg@10, up to 300 rounds with early stopping after 30 rounds without validation improvement) trained on 3,000 randomly sampled users. The feature vector for each (user, track) pair concatenates the 128-dimensional user-profile embedding, the 128-dimensional track-AE embedding, and 44 categorical/content track features: one-hot musical key (12), mode (2), and tempo class (9) encodings; smoothed target-encoded primary genre and artist name (2, fitted on training users only); and a multi-hot indicator of the top-20 most frequent instruments (20). The total feature dimension is thus $128 + 128 + 44 = 300$. The hybrid therefore fuses content (autoencoder) and behaviour (candidate generation) without using the graph structure, directly isolating the contribution of the GNN.

3.4. Reasoning and Semantic Inspection

Beyond representation learning, the OWL layer supports light-weight reasoning and transparent inspection. Aligning `mrc:Genre`, `mo:Instrument` and `mrc:Key` to their Wikidata super-classes and chaining release decades with `follows/followed by` lets a reasoner infer shared super-concepts and temporal adjacency that are not stated explicitly, which in turn densifies the bridges exploited by message passing. The enriched graph is loaded into a GraphDB triple store and queried with SPARQL to produce the qualitative evidence reported in Sec. 4 (e.g. genre rankings and per-entity neighbourhoods).

3.5. Evaluation Protocol

All recommenders are evaluated on a single, shared *user-level stratified* split of the interaction data (training / validation / test), stratified by song-level attributes (genre and release decade) so that the popularity and content distributions are preserved across folds; song features and graph structure are visible to every split, and only the interactions are partitioned, preventing leakage. Knowledge-graph embeddings and the autoencoder are unsupervised and therefore fit on the training data without a held-out ranking split.

We report a standard top- K suite—Recall@ K , Precision@ K , NDCG@ K , Hit-Rate@ K and MRR—together with two diversity-oriented measures, catalogue *Coverage* and mean *Popularity-Bias@K*, and a composite *Overall Score*

$$\text{Overall} = 0.6 \cdot \text{NDCG@}K + 0.2 \cdot \text{Coverage} + 0.2 \cdot (1 - \text{Popularity-Bias@}K),$$

which rewards models that are accurate *and* surface diverse, less popular content. Every model is scored with the same

Table 1. Top-10 test-set performance. Higher is better except Pop-Bias@10 ($\downarrow$). HGT row pending.

Model	Recall	NDCG	HitRate	MRR	Cov.	Overall
MostPopular	0.1151	0.0979	0.1841	0.0725	0.0020	0.1335
KNN-CF ($k=6$)	0.1036	0.1059	0.1704	0.0870	0.9154	0.4201
XGBoost-Hybrid	0.0134	0.0149	0.0257	0.0116	0.3698	0.2764
HGT (ours)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

recommendation lists sliced at $K \in \{5, 10, 20, 50, 100\}$. Pairwise differences are assessed with the paired Wilcoxon signed-rank test (with Cohen’s d_z effect sizes) and an omnibus Friedman test. Because full inference over all 284,864 test users is computationally prohibitive for the XGBoost model, pairwise and omnibus tests are conducted on a shared random subsample of 5,000 users across all models, ensuring a balanced, paired comparison.

4. Experiments and Results

4.1. Experimental Setup

After cold-start filtering (at least 5 interactions per user and 3 users per song) the modelling dataset contains 284,864 users and 6,054 tracks with their full jSymbolic feature vectors and knowledge-graph context. Interactions are partitioned with the user-level stratified split of Sec. 3.5: 1,943,681 training, 348,054 validation, and 559,806 test interactions, stratified by primary genre, release decade, key, mode, and tempo class. All models are trained on identical training interactions and evaluated on the held-out test interactions; reproducibility is fixed with seed 42. Hyperparameters are those listed in Sec. 3 and were selected on the validation split only.

4.2. Quantitative Comparison

Tab. 1 reports every completed model at $K = 10$ on the test set. The popularity prior is, as expected, a strong short-list accuracy baseline but collapses on diversity: it covers only 0.2% of the catalogue and exhibits the highest popularity bias (0.628). KNN-CF ($k=6$, selected by validation sweep) sacrifices a small amount of ranking accuracy for a dramatic gain in coverage (91.5%) and a large drop in popularity bias (0.133), which together push it to the best *Overall* score (0.420). The XGBoost two-stage hybrid sits between the two: it produces the lowest popularity bias of any model (0.032) and moderate catalogue coverage (37.0%), but its ranking accuracy is substantially lower (NDCG@10 = 0.015), reflecting the difficulty of content-only candidate generation when collaborative signal is sparse. **[TODO: Update the HGT row with final numbers once the HGT training run is complete; state explicitly whether the graph structure closes the accuracy gap over KNN-CF.]**

[TODO: Insert Recall@K / NDCG@K / Overall vs. K line plots (baseline-comparison cell).]

Figure 3. Metric vs. recommendation-list length K for all models.

Table 2. Friedman omnibus test and best-ranked model per metric.

Metric	χ^2	p	Best model
Recall@ K	[TODO:]	[TODO:]	[TODO:]
NDCG@ K	[TODO:]	[TODO:]	[TODO:]
HitRate@ K	[TODO:]	[TODO:]	[TODO:]
MRR	[TODO:]	[TODO:]	[TODO:]

Behaviour across list length. Fig. 3 sweeps the recommendation-list length $K \in \{5, \dots, 100\}$ for Recall@ K , NDCG@ K and the composite Overall score. For MostPopular and KNN-CF both accuracy and the composite score improve steadily with K . The XGBoost hybrid shows only modest gains in ranking accuracy as K grows (NDCG@100 = 0.025), but its Overall score rises from 0.270 at $K=5$ to 0.291 at $K=100$, driven by increasing catalogue coverage as the candidate pool expands. [TODO: Insert the multi- K comparison figure and add the HGT curve once available.]

Statistical significance. Pairwise paired Wilcoxon signed-rank tests are conducted on a shared random subsample of 5,000 test users across all models, ensuring a balanced, comparable sample; the omnibus Friedman test is reported in Tab. 2. [TODO: Populate Tab. 2 and the pairwise table from the comparison cell once the HGT model is available and all four models can be compared together.]

4.3. Learning Curves

Fig. 4 collects the training-time diagnostics for the completed models. The autoencoder converged cleanly, reaching a best training MSE of 0.616 before early stopping at epoch 155 (patience 15, max 200 epochs). The XGBoost model used all 300 boosting rounds and was still improving at round 300 (best validation ndcg@10 = 0.898), suggesting that additional rounds may further improve ranking accuracy; the smooth training curve shows no signs of overfitting. [TODO: Drop in the four PNGs exported to data/final/(ae_loss_curve.png, kge_loss_curve.png, hgt_training_curves.png,

[TODO: Insert learning curves: AE (MSE), KGE (loss), HGT (loss + val Recall/NDCG), XGBoost (ndcg@10 per round).]

Figure 4. Training and validation learning curves for the trained models.

[TODO: Insert population-qualitative mosaic and a per-user case study.]

Figure 5. Qualitative comparison of recommendation behaviour across models.

xgb_loss_curve.png) and comment on the KGE and HGT curves once those runs complete.]

4.4. Qualitative Analysis

We complement the metrics with three qualitative views. **(i) Population level:** per-model distributions of cosine-to-profile similarity and Jensen–Shannon divergence between the genre/mode/tempo/decade profiles of a user’s training history and their recommendations, plus dominant-attribute match rates (Fig. 5). **(ii) Per-user case studies:** successful users across different training-history sizes (light/medium/heavy), failure cases, and “mixed” users where models disagree, each shown with a cross-model metric table and train-vs-recommendation attribute distributions. **(iii) Latent space:** a UMAP/GMM projection of the learned node embeddings used to inspect cluster structure. [TODO: Insert the population-summary mosaic, one or two representative per-user case figures, and the latent-space projection; summarise where each model is strong or weak (e.g. popularity vs. niche taste, cold vs. rich profiles).]

4.5. Discussion

The completed baselines collectively illustrate the central tension of the task. **MostPopular** achieves the highest hit-rate on short lists (0.184 at $K=10$) but is nearly useless as

a discovery tool: it covers only 0.2% of the 6,054-song catalogue and assigns the highest popularity bias (0.628), confirming that it amplifies existing listening patterns rather than enabling exploration.

KNN-CF ($k=6$) demonstrates that collaborative filtering already breaks the popularity trap effectively: NDCG@10 rises slightly above MostPopular (0.106 vs. 0.098) while coverage jumps to 91.5% and popularity bias drops to 0.133, producing an Overall score of 0.420—more than three times the popularity baseline (0.134). The optimal neighbourhood size of 6 is notably small, suggesting that tight, high-quality neighbourhoods outperform broad ones for this dataset.

XGBoost-Hybrid reveals the limits of content-only candidate generation: despite incorporating musical features (key, mode, tempo, genre, instruments) and 128-dimensional autoencoder embeddings, the model’s ranking accuracy (NDCG@10 = 0.015) falls far below both collaborative baselines. The low accuracy likely reflects the mismatch between content similarity (which the AE embeddings capture) and the actual listening preferences of individual users, which are poorly approximated by a mean-pool user profile over only 3,000 training users. Notably, the model achieves the lowest popularity bias of any model (0.032), confirming that pure content-based retrieval naturally avoids recommending well-known tracks.

These three points of reference frame the expected contribution of the **HGT**: the graph-relational structure should enable message passing to carry both collaborative and content signals simultaneously, potentially closing the accuracy gap that the content-only hybrid exhibits while preserving the diversity improvements over the popularity baseline.

[TODO: Update this paragraph with the quantitative comparison once HGT training is complete.]

References

- [1] FirstName Alpher. Frobnication. *IEEE TPAMI*, 12(1):234–778, 2002. [5](#)
- [2] FirstName Alpher and FirstName Fotheringham-Smythe. Frobnication revisited. *Journal of Foo*, 13(1):234–778, 2003. [5](#)
- [3] FirstName Alpher, FirstName Fotheringham-Smythe, and FirstName Gamow. Can a machine frobnicate? *Journal of Foo*, 14(1):234–778, 2004. [5](#)
- [4] Thierry Bertin-Mahieux, Daniel P. W. Ellis, Brian Whitman, and Paul Lamere. The million song dataset. In *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR)*, 2011. [1](#)
- [5] Cory McKay, Julie Cumming, and Ichiro Fujinaga. Jsymbolic 2.2: Extracting features from symbolic music for use in musical and mir research. In *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, pages 348–354, 2018. [2](#)
- [6] Colin Raffel. *Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and*

Matching. PhD thesis, Columbia University, 2016. Includes the Lakh MIDI Dataset. [1](#)

- [7] The Echo Nest. Taste profile subset of the million song dataset, 2011. [1](#)

Appendix

A. Exploratory Overview of MSD Features

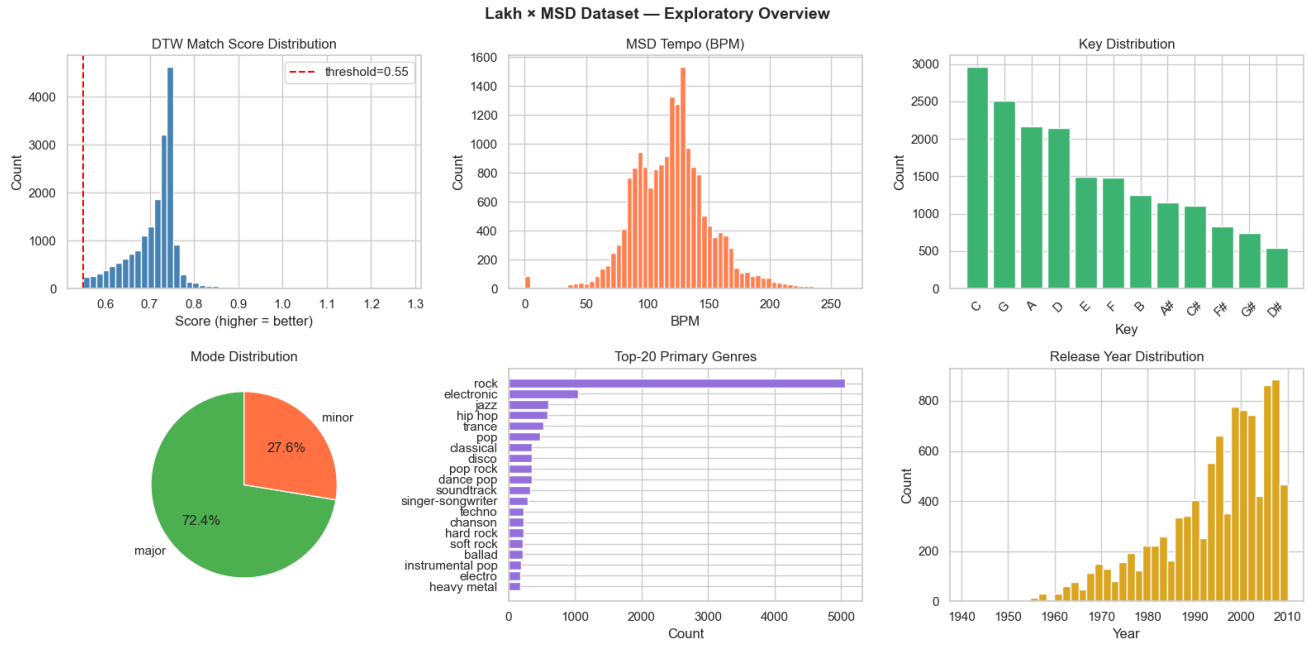


Figure 6. Exploratory Overview of MSD Features.

B. Exploratory Overview of User Taste Profile Filtering Pipeline and Features

Table 3. User dataset filtering pipeline statistics.

Step	Interactions	Unique Users	Unique Songs
Raw	48 373 586	1 019 318	384 546
– Mismatches	45 795 100	1 019 318	378 309
$\cap$ LMD	4 243 384	848 243	7465
– Cold-start	3 048 959	299 156	7227

Echo Nest Taste Profile — Filtered Dataset Diagnostics

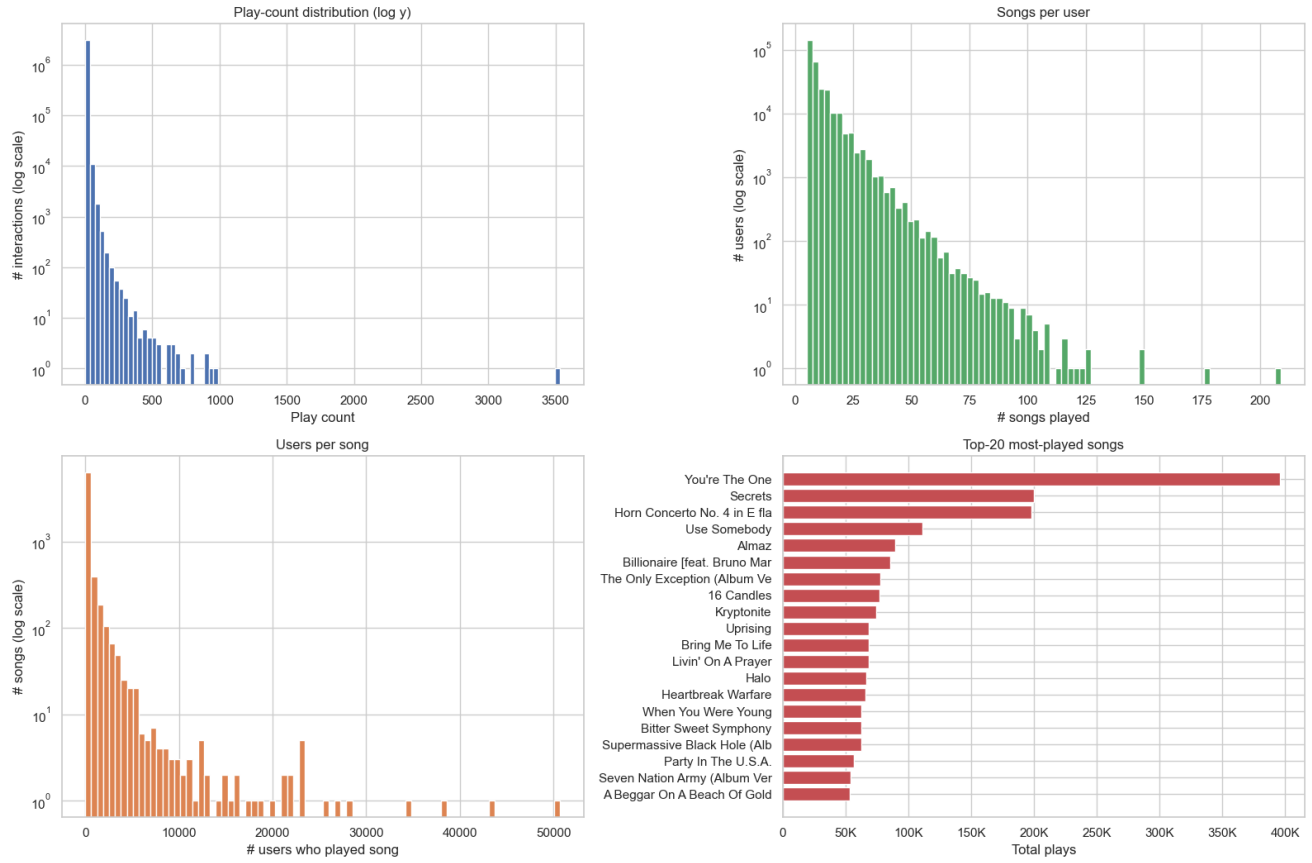


Figure 7. Descriptive statistics of the filtered Echo Nest Taste Profile dataset, including the play-count distribution, songs per user, users per song, and the top-20 most-played songs.

C. Music-related feature inventory in simple knowledge graph

Table 4. Music features from MSD and jSymbolic mapping to RDF representation

Feature Name	Description	Origin	RDF Term	Storage Type
track_id	Unique MSD track identifier used as the primary URI for the track node	MSD	mo:uuid (literal on track node)	Literal (xsd:string)
song_id	Echo Nest song identifier used for cross-referencing with taste profile	MSD	dct:identifier (literal on track node)	Literal (xsd:string)
artist_id	MSD canonical artist identifier used as the artist node URI	MSD	URI slug (artist: namespace)	Named Node (mo:Performer)
artist_mbid	MusicBrainz cross-reference identifier for the artist	MSD	mo:musicbrainz_guid	Literal (xsd:string)
artist_name	Display name of the artist	MSD	foaf:name	Literal (xsd:string)
title	Title of the track	MSD	dct:title	Literal (xsd:string)
album_name	Name of the release/album the track belongs to	MSD	dct:isPartOf	Literal (xsd:string)
year	Release year of the track	MSD	dct:date	Literal (xsd:gYear)
key	Musical key of the track (C C# D ... B)	MSD	mrc:hasKey	Named Node (mrc:Key individual)
key_confidence	Confidence score of the key detection (0–1)	MSD	mrc:keyConfidence	Literal (xsd:double)
mode	Modality of the track (Major or Minor)	MSD	mrc:hasMode	Named Node (mrc:Mode individual)
mode_confidence	Confidence score of the mode detection (0–1)	MSD	mrc:modeConfidence	Literal (xsd:double)
loudness	Overall loudness of the track in decibels	MSD	mrc:loudness	Literal (xsd:double)
duration	Duration of the track in seconds	MSD	mo:duration	Literal (xsd:double)
primary_genre	Primary genre tag of the artist (weight = 1.0)	MSD	mrc:hasGenre (via artist)	Named Node (mrc:Genre individual)
top3.genres	Top 3 genre tags of the artist with rank-based weights	MSD	mrc:hasGenre (via artist)	Named Node (mrc:Genre individual)
artist_terms	Full list of MSD genre/style tags with published weights	MSD	mrc:hasGenre (via artist)	Named Node (mrc:Genre individual)
midi_instrument_names	List of General MIDI instrument names present in the MIDI file	MIDI (LMD)	mo:instrument (via performance)	Named Node (mo:Instrument individual)
tempo (Mean.Tempo / msd.tempo)	Mean BPM of the track; used to derive the categorical tempo class	jSymbolic / MSD	mo:tempo	Literal (xsd:double)
Tempo class	Categorical tempo marking derived from BPM (e.g. Allegro Andante)	Derived	mrc:hasTempoClass	Named Node (mrc:TempoClass individual)
Mean.Tempo	Mean tempo computed by jSymbolic	jSymbolic	mrc:meanTempo	Literal (xsd:double)
Initial.Tempo	Tempo at the start of the piece	jSymbolic	mrc:initialTempo	Literal (xsd:double)
Tempo.Variability	Variability of tempo across the piece	jSymbolic	mrc:tempoVariability	Literal (xsd:double)
Note.Density	Average number of notes per second	jSymbolic	mrc:noteDensity	Literal (xsd:double)
Average.Note.Duration	Mean duration of individual notes	jSymbolic	mrc:averageNoteDuration	Literal (xsd:double)
Amount.of.Staccato	Proportion of notes that are staccato	jSymbolic	mrc:amountOfStaccato	Literal (xsd:double)
Amount.of.Arpeggiation	Degree of arpeggiated melodic motion	jSymbolic	mrc:amountOfArpeggiation	Literal (xsd:double)
Chromatic.Motion	Proportion of melodic intervals that are semitones	jSymbolic	mrc:chromaticMotion	Literal (xsd:double)
Chord.Duration	Mean duration of chords	jSymbolic	mrc:chordDuration	Literal (xsd:double)
Average.Number.of.Independent.Voices	Mean number of simultaneous melodic voices	jSymbolic	mrc:averageNumberOfIndependentVoices	Literal (xsd:double)
Average.Note.to.Note.Change.in.Dynamics	Mean change in dynamics between consecutive notes	jSymbolic	mrc:averageNoteToNoteChangeInDynamics	Literal (xsd:double)